Simple Web Application Vulnerability Assessment

1. Introduction

The objective of this project is to assess the security of a vulnerable web application using common web application security testing techniques. We have set up a vulnerable web application called **DVWA (Damn Vulnerable Web Application)** in a Docker container. The goal is to identify and exploit basic vulnerabilities, such as **SQL Injection** and **Cross-Site Scripting (XSS)**, in the application and document findings for further remediation.

2. Setting Up the Vulnerable Web Application on Windows

We have set up **DVWA** using Docker on Windows. The steps to set up DVWA on Docker are:

1. **Install Docker Desktop**:
    - Download and install Docker Desktop from Docker's official website.
    - After installation, make sure Docker Desktop is running.
2. **Pull the DVWA Image**: Open **Command Prompt** or **PowerShell** and run the following command:

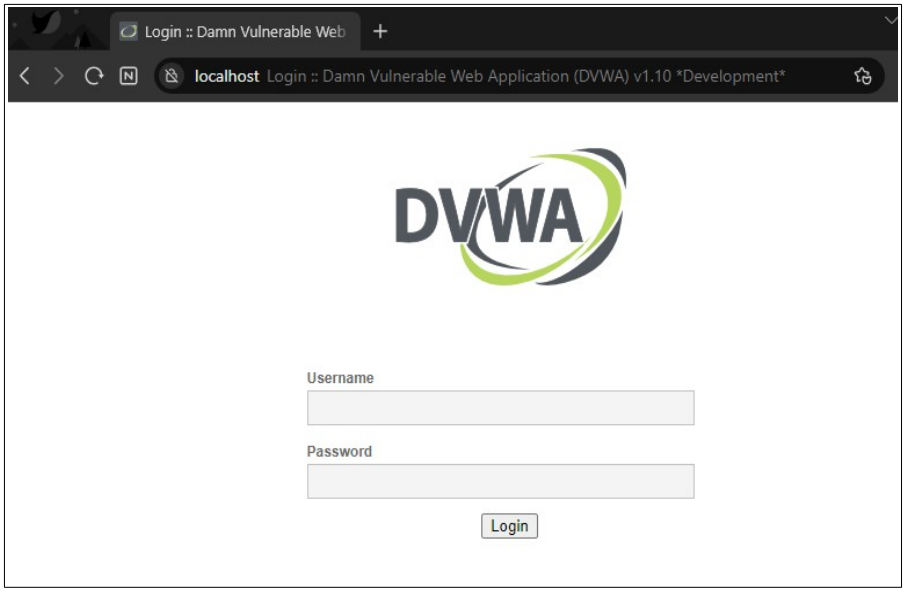
    ```
    docker pull vulnerables/web-dvwa
    ```

3. **Run DVWA in Docker**: In **Command Prompt** or **PowerShell**, run the following command:

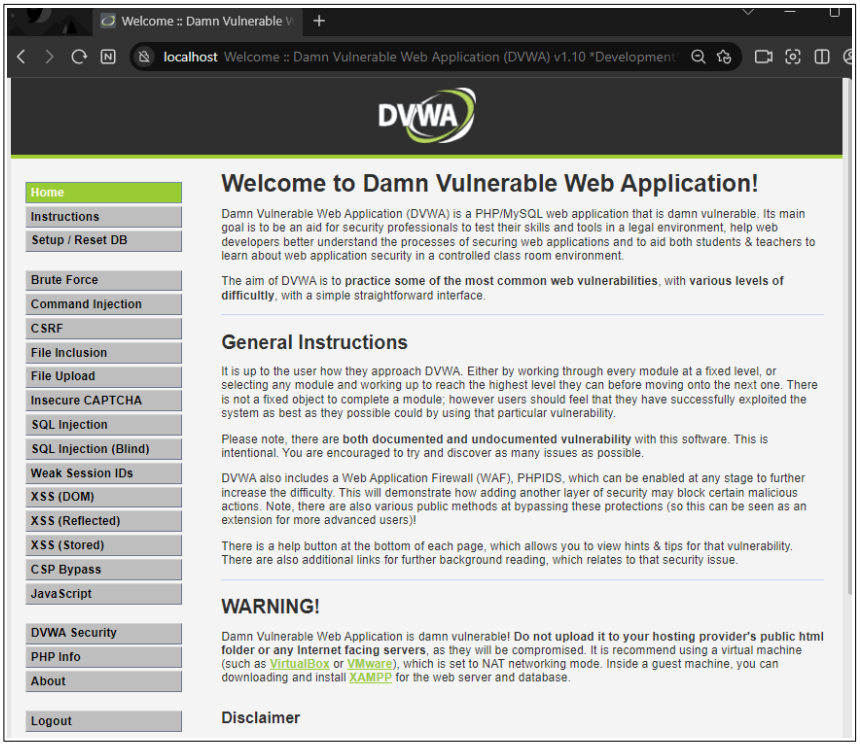
    ```
    docker run --rm -d -p 80:80 vulnerables/web-dvwa
    ```

    This command will start DVWA on port **80**.

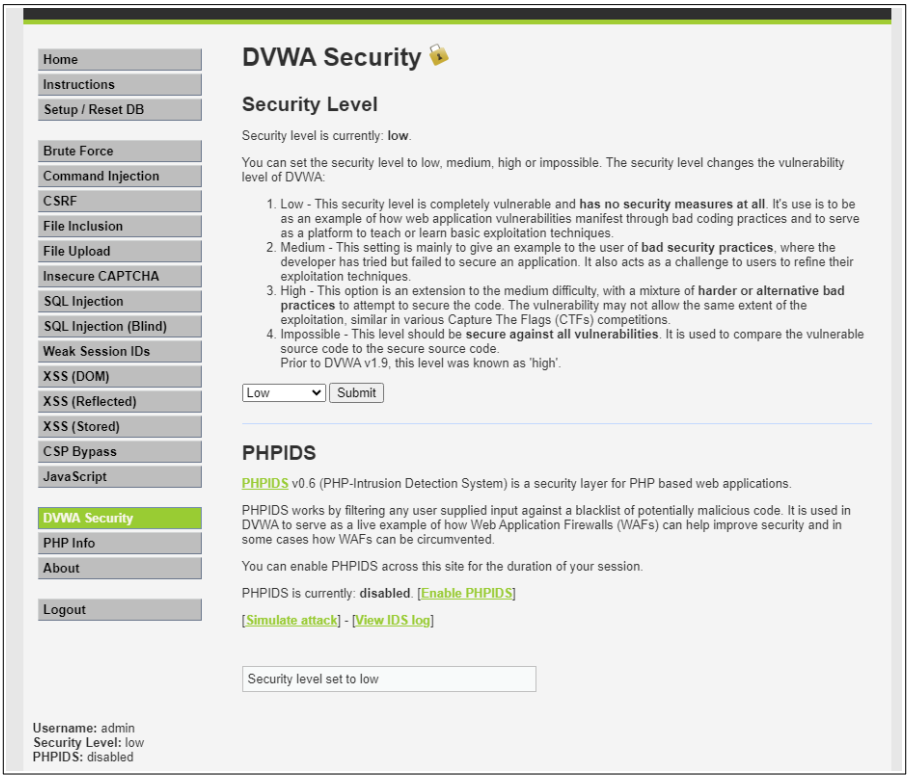
4. **Access DVWA**: Open **your web browser** (e.g., Google Chrome or Firefox) and visit `http://localhost`.



5. **Login to DVWA**:
    - **Username**: `admin`
    - **Password**: `password`



6. **Set Security Level**: Once logged in, go to the **DVWA Security** tab and set the security level to **Low** for easier vulnerability testing.

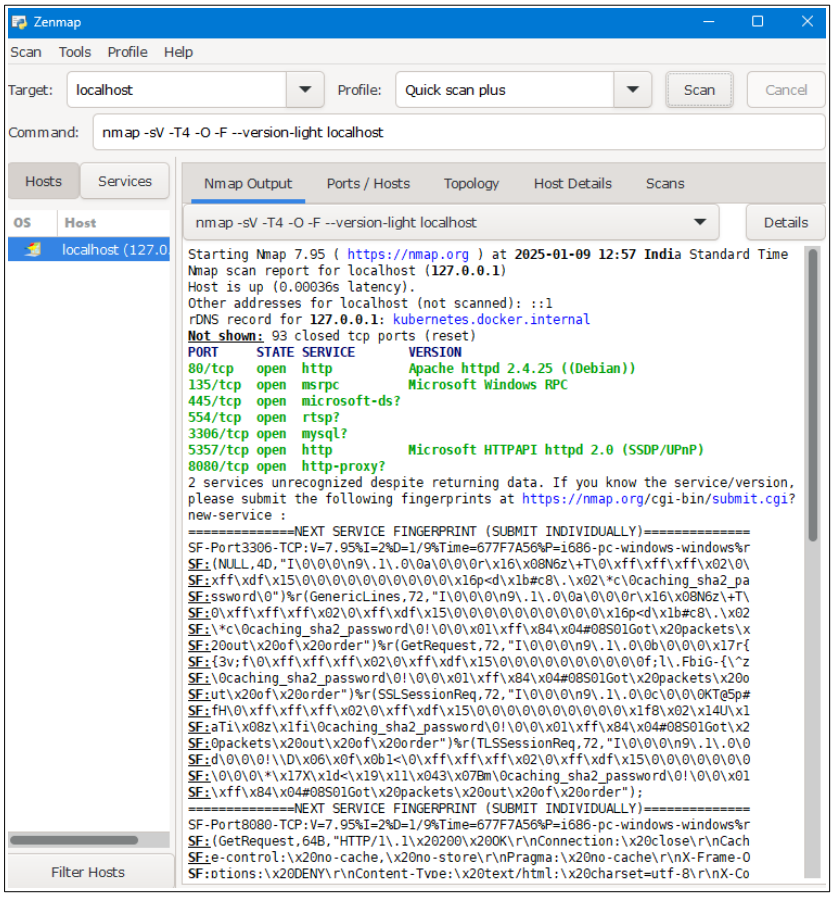


3. Reconnaissance with Nmap

1. **Install Nmap**:
    - Download the installer from Nmap's official website.
    - Follow the installation instructions.
2. **Run Nmap Scan**: Open **Command Prompt** and run:

    ```
    nmap -sV localhost
    ```

3. **Results**:
    - The Nmap scan reveals open port **80** with the **Apache HTTPD** service running.



4. Identifying Vulnerabilities

We performed tests for **SQL Injection** and **XSS** using the **DVWA** application.

4.1 SQL Injection

1. **Test for SQL Injection**:
    - Go to the **DVWA login page** (`http://localhost`).
    - Enter the following in the **Username** field:

        ```
        admin' OR 1=1 --
        ```

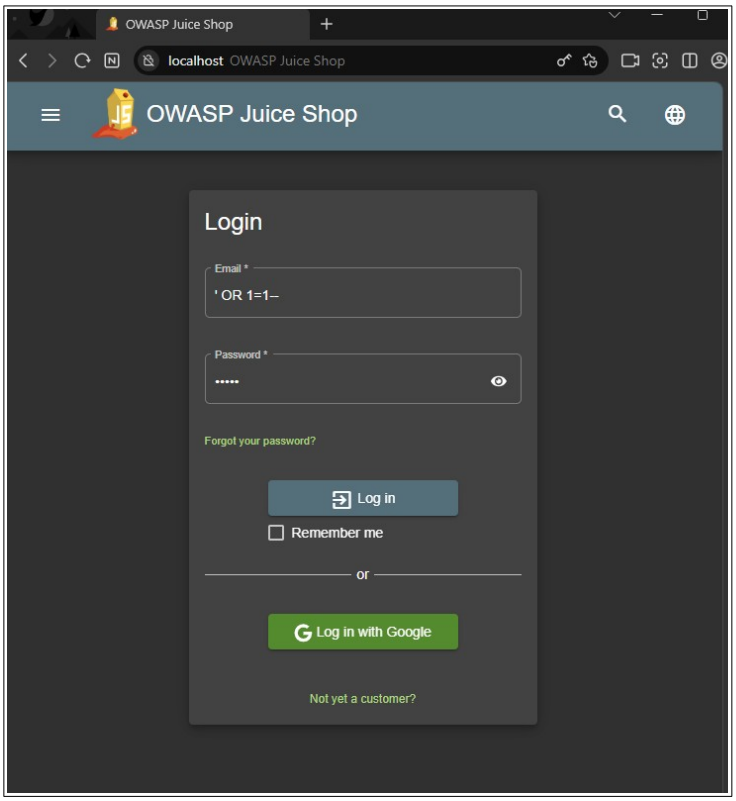
    - Enter anything in the **Password** field (e.g., `anything`).

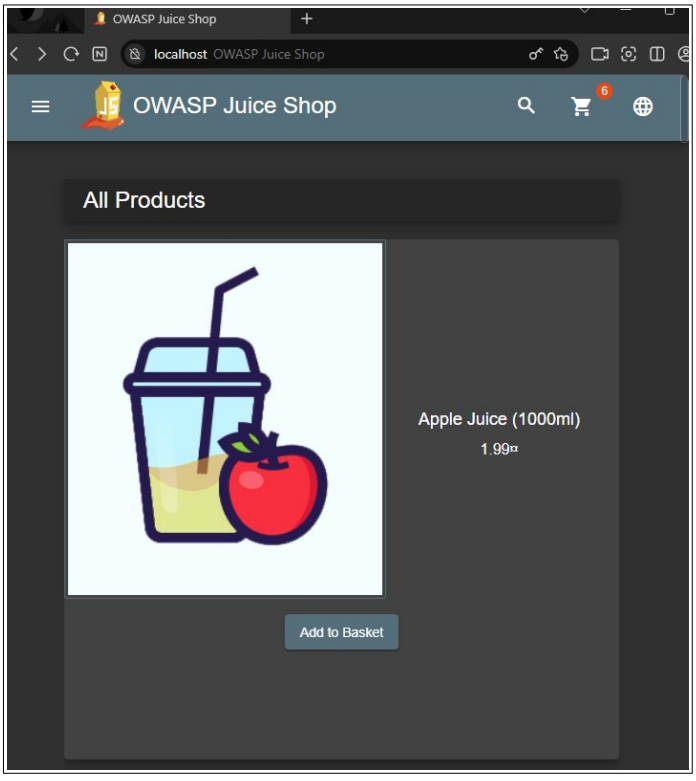
2. **Result**:

- The query becomes:

```
SELECT * FROM users WHERE username = 'admin' OR 1=1 --' AND password
= 'anything'
```

- Since `1=1` is always true, this bypasses the login authentication, allowing access.



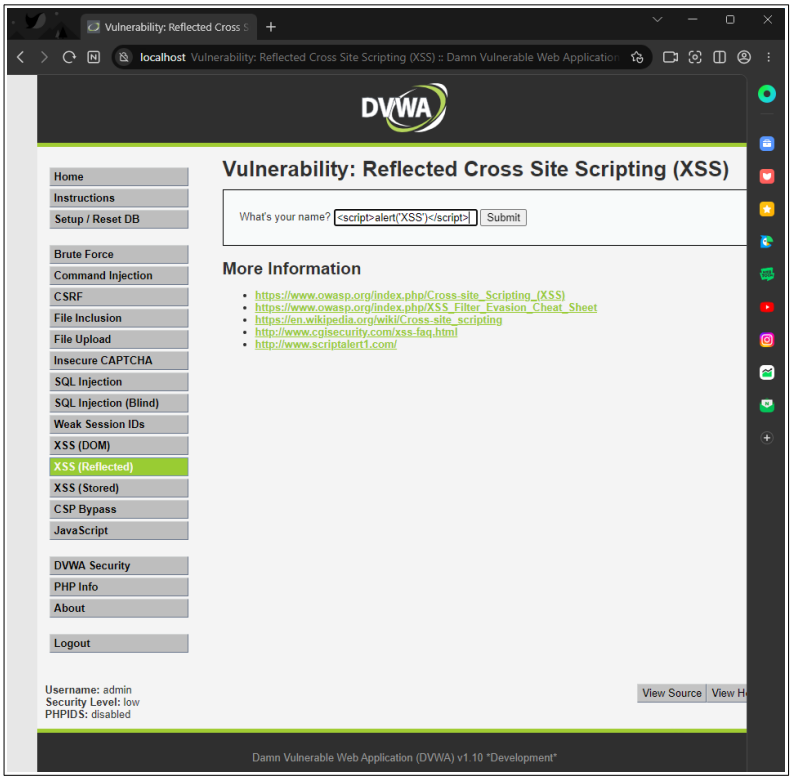


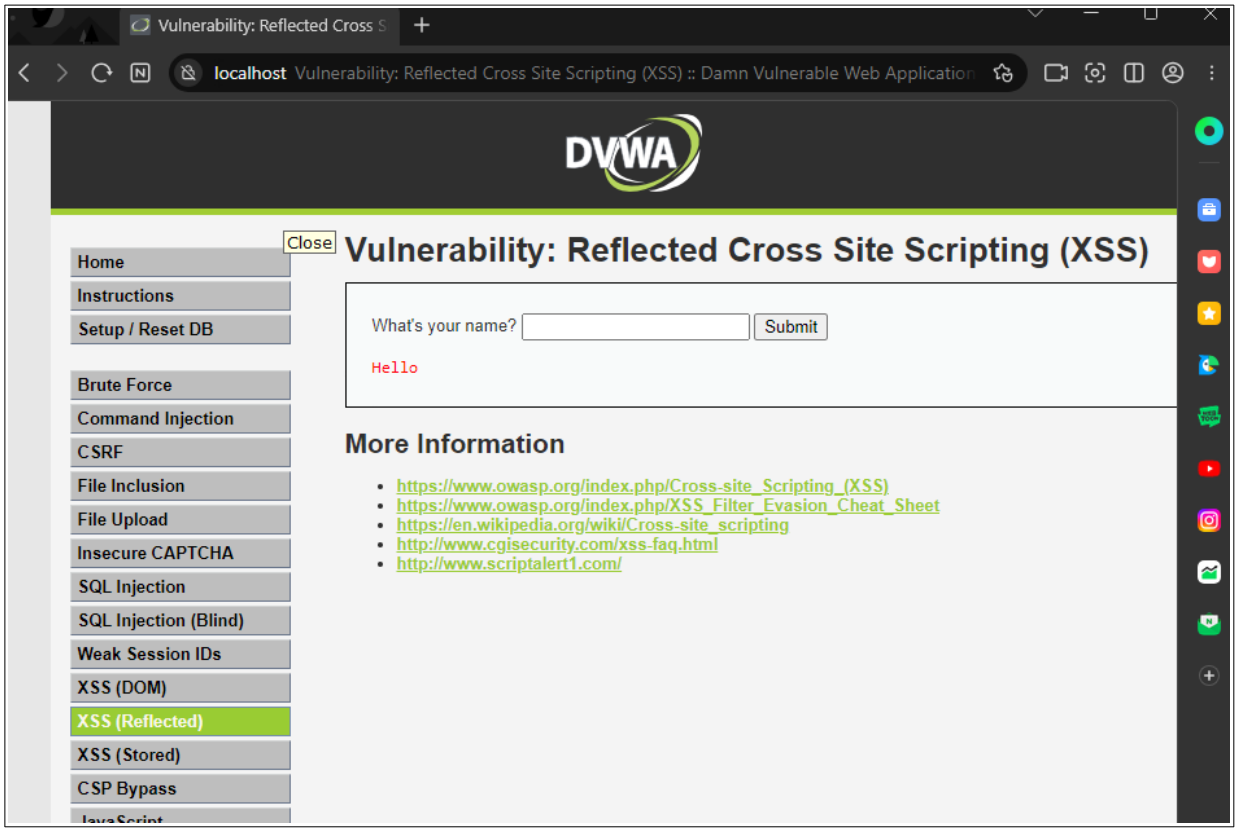
4.2 Cross-Site Scripting (XSS)

1. **Test for XSS**:
    - Go to the **XSS (Reflected)** page in DVWA.
    - In the input field, enter the following XSS payload:

      ```
      <script>alert('XSS');</script>
      ```

2. **Result**:
    - The injected JavaScript is executed, displaying an alert box with "XSS".





5. Conclusion

In this project, we identified and exploited two major vulnerabilities in the **DVWA** application on Windows:

- **SQL Injection**: Allowed us to bypass authentication by injecting a SQL query into the login form.
- **Cross-Site Scripting (XSS)**: Allowed us to inject and execute JavaScript code in the browser.

6. Recommendations

- **SQL Injection**:
    - Use **prepared statements** and **parameterized queries** to prevent SQL injection.
    - Sanitize and validate user inputs to avoid direct execution of malicious SQL queries.
- **Cross-Site Scripting (XSS)**:
    - Use proper **output encoding** and **input sanitization** to prevent the execution of JavaScript in user input fields.
    - Consider implementing **Content Security Policy (CSP)** to limit script execution.